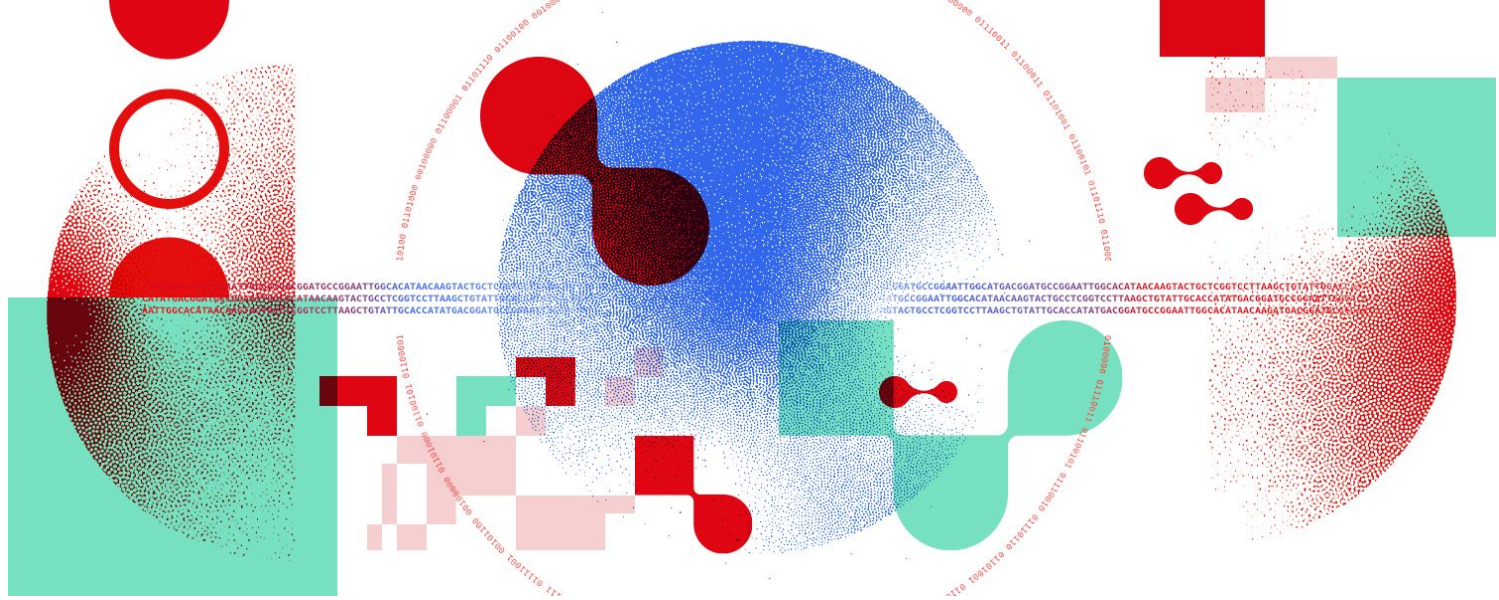




# Snakemake for reproducible research

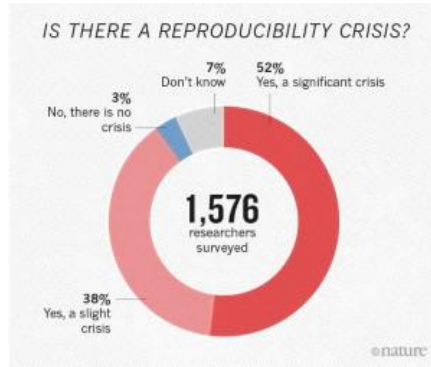
## Introduction to Snakemake

# What is reproducibility?

- Key component of the scientific method
  - Replicability: new data + same methods = same results
  - **Reproducibility: same data + same analysis = same results**
  - Repeatability: same researchers + same study + repeated experiments = same results
- "Reproducibility is more or less the ability to draw similar conclusions from replicates studies"
  - Diaba-Nuhoho, P., Amponsah-Offeh, M., *BMC Research Notes* (2021), <https://doi.org/10.1186/s13104-021-05875-3>

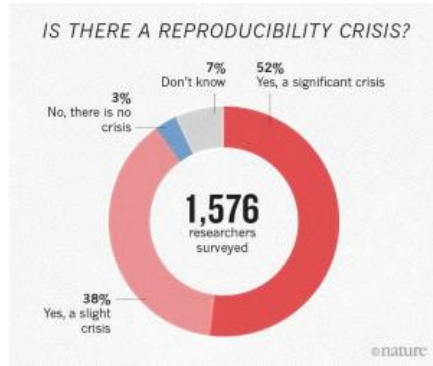
# Is there a reproducibility crisis?

- Baker, M., *Nature* (2016),  
<https://doi.org/10.1038/533452a>
  - ~1600 researchers: 52% significant crisis, 38% slight crisis (90% in total)

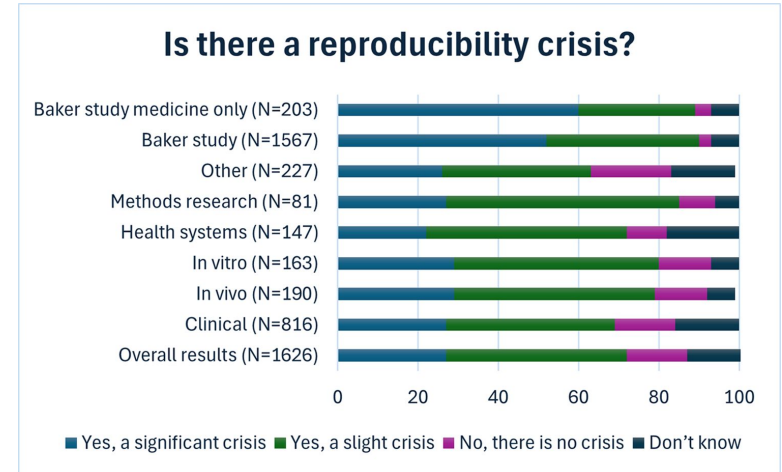


# Is there a reproducibility crisis?

- Baker, M., *Nature* (2016),  
<https://doi.org/10.1038/533452a>
  - ~1600 researchers: 52% significant crisis, 38% slight crisis (90% in total)

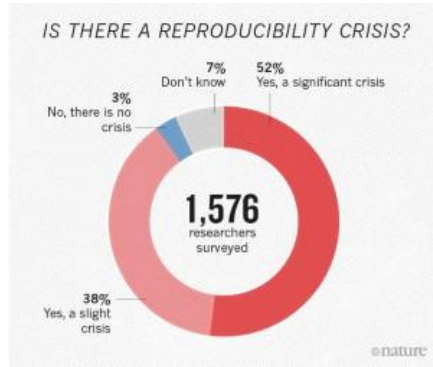


- Cobey, K. et al., *Plos Biology* (2024),  
<https://doi.org/10.1371/journal.pbio.3002870>
  - 27% significant crisis in biomedicine, 45 % slight crisis (72% in total)

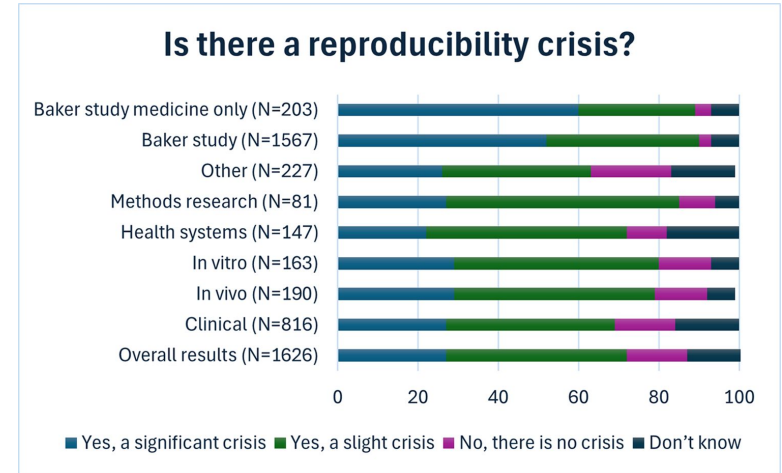


# Is there a reproducibility crisis?

- Baker, M., *Nature* (2016), <https://doi.org/10.1038/533452a>
  - ~1600 researchers: 52% significant crisis, 38% slight crisis (90% in total)



- Cobey, K. et al., *Plos Biology* (2024), <https://doi.org/10.1371/journal.pbio.3002870>
  - 27% significant crisis in biomedicine, 45 % slight crisis (72% in total)



- Han, H., *BMC Medical Genomics* (2025), <https://doi.org/10.1186/s12920-024-02072-6>
  - AI is revolutionizing biomedical data science at a fast pace but also brings a lot of questions about reproducibility

# Why is that?

- Absence of knowledge/infrastructure
- Questionable research practices and fraud
- Statistical issues
  - Low statistical power
  - Statistical heterogeneity
- Publication system in science
  - Publication bias (non-significant results/unoriginal replications not published)
  - "Publish or perish"
  - Standards of reporting, open-access

## COMPLETED

50 experiments

## INITIATED

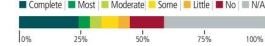
87 experiments

## DESIGNED

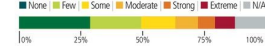
193 experiments

### BARRIERS

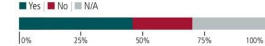
#### Modifications implemented



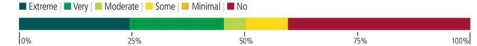
#### Modifications needed



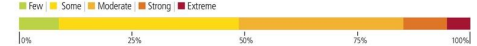
#### Reagents shared



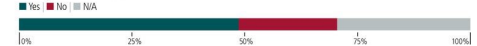
#### Authors helped



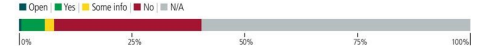
#### Protocol clarifications needed



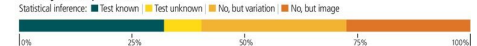
#### Reagents offered



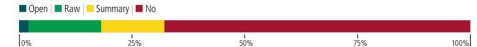
#### Code shared



#### Analysis reported



#### Data shared



Errington, T.M. *et al.*, *eLife* (2021),  
<https://doi.org/10.7554/eLife.67995>

# Why is that?

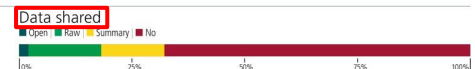
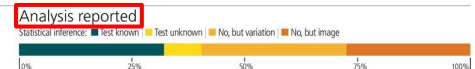
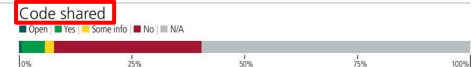
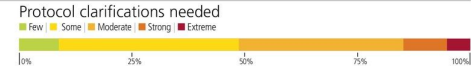
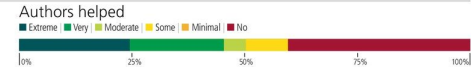
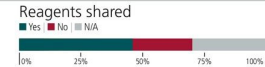
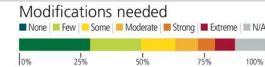
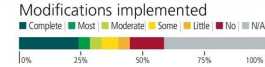
- Absence of knowledge/infrastructure
- Questionable research practices and fraud
- Statistical issues
  - Low statistical power
  - Statistical heterogeneity
- Publication system in science
  - Publication bias (non-significant results/unoriginal replications not published)
  - "Publish or perish"
  - Standards of reporting, open-access

COMPLETED  
50 experiments

INITIATED  
87 experiments

DESIGNED  
193 experiments

## BARRIERS



Errington, T.M. *et al.*, *eLife* (2021),  
<https://doi.org/10.7554/eLife.67995>

# What do Workflow Management Systems (WMS) bring?

- WMS can solve several hidden reproducibility issues:
  - Entirely:
    - OS
    - Version
    - Language
    - Readability
    - Availability
  - Partially (at least):
    - File format
    - Metadata
    - Parameters/options

# What is the idea behind WMS?

- Provide reproducible and scalable infrastructures to set-up, perform, and monitor a defined sequence of tasks

# What is the idea behind WMS?

- Provide reproducible and scalable infrastructures to set-up, perform, and monitor a defined sequence of tasks

|                                         | <u><a href="#">Nextflow</a></u> | <u><a href="#">Snakemake</a></u> | <u><a href="#">Galaxy</a></u> |
|-----------------------------------------|---------------------------------|----------------------------------|-------------------------------|
| Language<br>(How to code the workflow?) | Groovy (~ Java)                 | Extension of Python              | Java + Python                 |

# What is the idea behind WMS?

- Provide reproducible and scalable infrastructures to set-up, perform, and monitor a defined sequence of tasks

|                                                        | <u>Nextflow</u>         | <u>Snakemake</u>                                                     | <u>Galaxy</u>                                                                 |
|--------------------------------------------------------|-------------------------|----------------------------------------------------------------------|-------------------------------------------------------------------------------|
| Language<br>(How to code the workflow?)                | Groovy (~ Java)         | Extension of Python                                                  | Java + Python                                                                 |
| Execution system<br>(How are tasks organised and run?) | Join processes manually | Automatically resolve dependencies from last outputs to first inputs | Web-based GUI, manual task organisation but no need for programming knowledge |

# What is the idea behind WMS?

- Provide reproducible and scalable infrastructures to set-up, perform, and monitor a defined sequence of tasks

|                                                        | <u>Nextflow</u>         | <u>Snakemake</u>                                                     | <u>Galaxy</u>                                                                 |
|--------------------------------------------------------|-------------------------|----------------------------------------------------------------------|-------------------------------------------------------------------------------|
| Language<br>(How to code the workflow?)                | Groovy (~ Java)         | Extension of Python                                                  | Java + Python                                                                 |
| Execution system<br>(How are tasks organised and run?) | Join processes manually | Automatically resolve dependencies from last outputs to first inputs | Web-based GUI, manual task organisation but no need for programming knowledge |

# Some interesting features of Snakemake



- **Python-based:** concise and readable = user-friendly

# Some interesting features of Snakemake



- **Python-based**: concise and readable = user-friendly
- Easily deployable/executable **locally** or **remotely** (computation clusters and clouds)

# Some interesting features of Snakemake



- **Python-based**: concise and readable = user-friendly
- Easily deployable/executable **locally** or **remotely** (computation clusters and clouds)
- Integrated package management via **conda/mamba** (package manager) and **apptainer** (container manager)

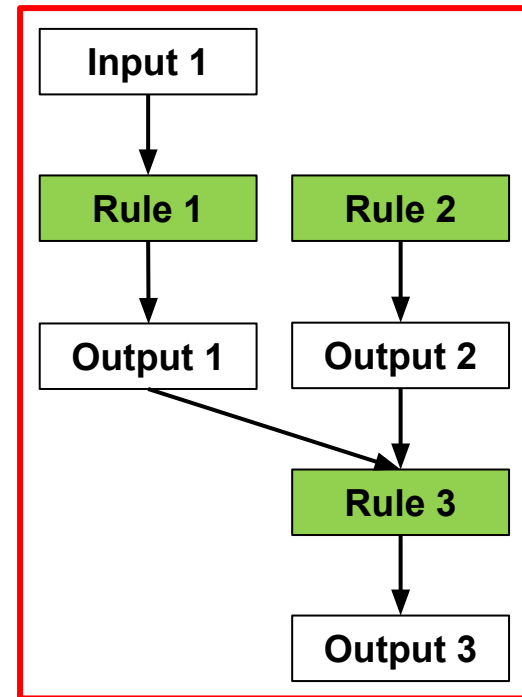
# Some interesting features of Snakemake



- **Python-based**: concise and readable = user-friendly
- Easily deployable/executable **locally** or **remotely** (computation clusters and clouds)
- Integrated package management via **conda/mamba** (package manager) and **apptainer** (container manager)
- Once you have downloaded a workflow, it is easy to:
  - Run Snakemake in a **strictly controlled environment** (OS, software, versions, parameters...)
  - Efficiently and automatically reproduce analyses, results and figures

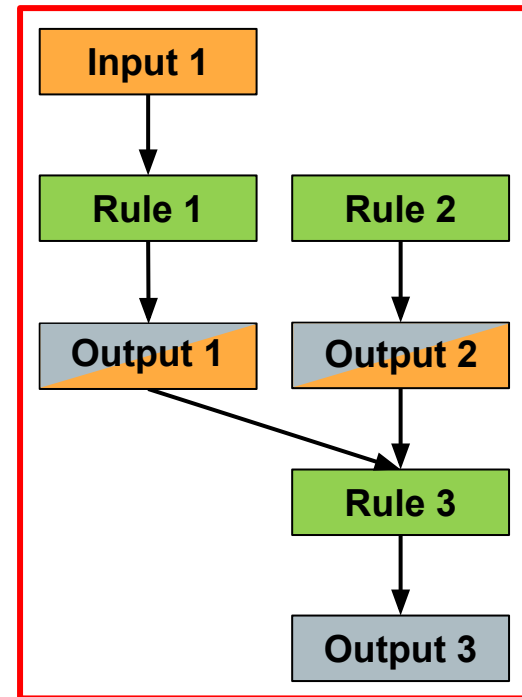
# How does Snakemake work?

- **Workflow:**
  - Network of dependent **rules**



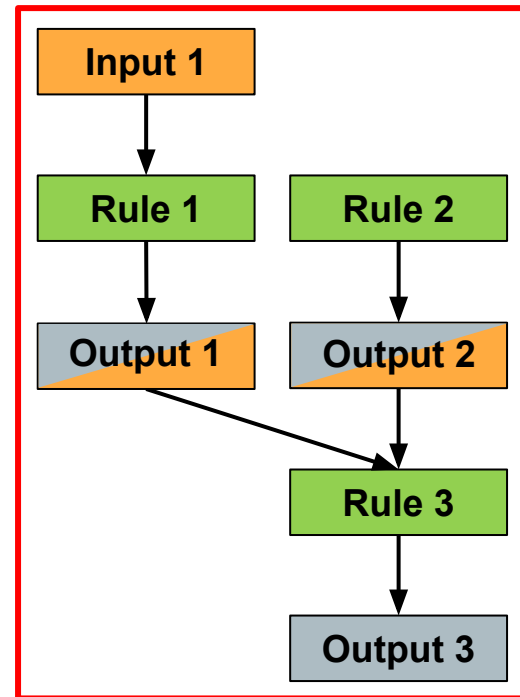
# How does Snakemake work?

- **Workflow:**
  - Network of dependent **rules**
- **Rule:**
  - Smallest part of a workflow
  - **Set of instructions** to create one or more **output(s)** from zero or more **input(s)**



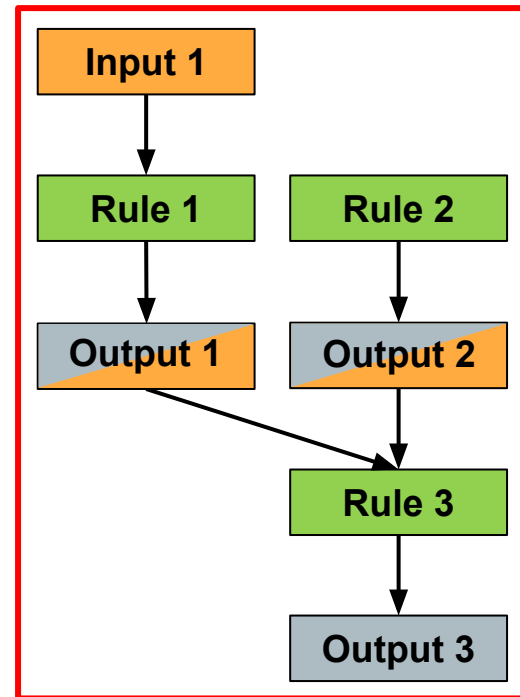
# How does Snakemake work?

- **Workflow:**
  - Network of dependent **rules**
- **Rule:**
  - Smallest part of a workflow
  - **Set of instructions** to create one or more **output(s)** from zero or more **input(s)**
- **Job:**
  - **Execution** of a **rule** with **specific input(s)/output(s)**
  - Success conditions:
    - **No error**
    - **All expected outputs present**



# How does Snakemake work?

- **Workflow:**
  - Network of dependent **rules**
- **Rule:**
  - Smallest part of a workflow
  - **Set of instructions** to create one or more **output(s)** from zero or more **input(s)**
- **Job:**
  - **Execution** of a **rule** with **specific input(s)/output(s)**
  - Success conditions:
    - **No error**
    - **All expected outputs present**
- Directed Acyclic Graph (DAG) determined from the required outputs.



# Why is a DAG useful?

- Add samples/inputs to existing analyses without re-running everything
- Skip parts of the DAG to avoid recomputing
  - Resume a workflow that failed part-way
  - Save time and resources (CPU, memory, energy, money)

# Why is a DAG useful?

- Visualise workflow structure and rule/file dependencies
- Simplify debugging
- Avoid needless recomputation by skipping parts of the DAG
  - Add samples/inputs to existing analyses
  - Resume workflow that failed part-way
  - Save time and resources (CPU, memory, energy, money)

# What does Snakemake really look like?

```
rule rename_file:
    input:
        'data/file.txt'
    output:
        'results/renamed_file.txt'
    shell:
        'mv data/file.txt results/file_renamed.txt'
```

# Snakemake lingo cheatsheet

- Snakemake keyword
- Rule name (user-defined)
- Snakemake directives
- Directives values:
  - Object
  - String (file path)
  - Instruction (command)
  - Numeric values (seen later)
- Wildcards
- Placeholders

```
rule rename_file:  
    input:  
        'data/{file}.txt'  
    output:  
        'results/{file}.txt'  
    shell:  
        'mv {input} {output}'
```

# Exercises

- Throughout the day:
  - Develop a simple RNAseq analysis workflow, from reads (fastq files) to Differentially Expressed Genes (DEG)
- For this session:
  - Understand the structure of a Snakemake workflow
  - Create your first rules and Snakefile
  - Chain rules together
  - Run your first workflow

